

# YouTube → Blog Post Agent

Complete Project Summary  
Generated: June 23, 2026

This document is an authoritative reference for the yt-agent2 project — a Streamlit web application that converts any YouTube video into a publish-ready blog post using Groq's Whisper API for transcription and Llama 3.3 70B for writing. The following sections cover the complete folder structure and the full contents and purpose of every source file.

## 1 · Folder Structure

|                     |                                             |
|---------------------|---------------------------------------------|
| yt-agent2/          |                                             |
| agents/             |                                             |
| transcript_agent.py | ← Agent 1: downloads audio & transcribes    |
| blog_post_agent.py  | ← Agent 2: researches & writes blog post    |
| __pycache__/        | (auto-generated, not committed)             |
| .env                | ← Secret keys (not committed to git)        |
| .env.example        | ← Template showing required env vars        |
| app.py              | ← Streamlit UI entry point                  |
| nixpacks.toml       | ← Railway: install ffmpeg system dependency |
| orchestrator.py     | ← Wires the two agents together             |
| Procfile            | ← Railway: process definition               |
| railway.toml        | ← Railway: build & deploy configuration     |
| requirements.txt    | ← Python package dependencies               |

## 2 · File Reference

### agents/transcript\_agent.py

Location  
yt-agent2/agents/transcript\_agent.py

Purpose  
Agent 1 of the pipeline. Accepts a YouTube URL, downloads its audio track using yt-dlp (converted to MP3 at 64 kbps via ffmpeg to stay within Groq Whisper's 25 MB file-size limit), then sends the audio to Groq's whisper-large-v3-turbo model and returns the plain-text transcript. Contains standalone `__main__` test that can be run directly.

Full Contents  
"""

TranscriptAgent

-----

Responsibility: Given a YouTube URL, download the audio and return a plain-text transcript using Groq's Whisper API.

Dependencies: yt-dlp, groq, python-dotenv  
"""

```
import os
import tempfile
import yt_dlp
from groq import Groq
from dotenv import load_dotenv
```

load\_dotenv()

class TranscriptAgent:

"""

Agent 1 – Downloads YouTube audio and transcribes it via Groq Whisper.

Usage:

```
agent = TranscriptAgent()
transcript = agent.run("https://www.youtube.com/watch?v=...")
"""
```

MAX\_FILE\_SIZE\_MB = 24

```
def __init__(self):
    api_key = os.getenv("GROQ_API_KEY")
    if not api_key:
        raise EnvironmentError(
            "GROQ_API_KEY not found. Add it to your .env file."
        )
    self.client = Groq(api_key=api_key)
    print("[TranscriptAgent] Initialised ✓")

def run(self, youtube_url: str) -> str:
    self._validate_url(youtube_url)
    print(f"[TranscriptAgent] Processing: {youtube_url}")
    with tempfile.TemporaryDirectory() as tmp_dir:
        audio_path = self._download_audio(youtube_url, tmp_dir)
        transcript = self._transcribe(audio_path)
    print(f"[TranscriptAgent] Done – {len(transcript)} characters transcribed.")
    return transcript

def _download_audio(self, url: str, output_dir: str) -> str:
    output_template = os.path.join(output_dir, "audio.%(ext)s")
    ydl_opts = {
        "format": "worstaudio/bestaudio",
        "outtmpl": output_template,
        "postprocessors": [{
            "key": "FFmpegExtractAudio",
            "preferredcodec": "mp3",
            "preferredquality": "64",
        }],
        "quiet": True,
        "no_warnings": True,
    }
    print("[TranscriptAgent] Step 1/2 – Downloading audio ...")
    try:
        with yt_dlp.YoutubeDL(ydl_opts) as ydl:
            ydl.download([url])
    except yt_dlp.utils.DownloadError as e:
        raise RuntimeError(f"Audio download failed: {e}") from e
    audio_path = self._find_audio_file(output_dir)
```

```

size_mb = os.path.getsize(audio_path) / (1024 * 1024)
print(f"[TranscriptAgent] Audio downloaded → {size_mb:.1f} MB")
if size_mb > self.MAX_FILE_SIZE_MB:
    raise RuntimeError(
        f"Audio file is {size_mb:.1f} MB, exceeds Groq Whisper's 25 MB limit."
    )
return audio_path

def _find_audio_file(self, directory: str) -> str:
    audio_extensions = {".mp3", ".m4a", ".ogg", ".opus", ".wav", ".webm"}
    for filename in os.listdir(directory):
        if os.path.splitext(filename)[1].lower() in audio_extensions:
            return os.path.join(directory, filename)
    raise RuntimeError(
        "No audio file found after download. Check that ffmpeg is installed."
    )

def _transcribe(self, audio_path: str) -> str:
    print("[TranscriptAgent] Step 2/2 – Transcribing via Groq Whisper ...")
    try:
        with open(audio_path, "rb") as audio_file:
            response = self.client.audio.transcriptions.create(
                file=audio_file,
                model="whisper-large-v3-turbo",
                response_format="text",
                language="en",
            )
    except Exception as e:
        raise RuntimeError(f"Groq Whisper transcription failed: {e}") from e
    transcript = str(response).strip()
    if not transcript:
        raise RuntimeError("Whisper returned an empty transcript.")
    return transcript

def _validate_url(self, url: str) -> None:
    if not url or not isinstance(url, str):
        raise ValueError("YouTube URL must be a non-empty string.")
    url = url.strip()
    if "youtube.com" not in url and "youtu.be" not in url:
        raise ValueError(f"'{url}' does not look like a YouTube URL.")

```

## agents/blog\_post\_agent.py

---

### Location

yt-agent2/agents/blog\_post\_agent.py

### Purpose

Agent 2 of the pipeline. Makes two sequential LLM calls to Groq's llama-3.3-70b-versatile model: (1) a research pass that analyses the transcript and produces structured internal notes covering core topic, target audience, key insights, hook angle, section structure, human element, and closing direction; (2) a writing pass that uses those notes plus the original transcript to produce a polished 600-900 word publish-ready blog post in Markdown. The research notes are never exposed to the user.

### Full Contents

```
"""
```

```
BlogPostAgent
```

```
-----
```

```
Responsibility: Given a plain-text transcript, run an internal
research pass and then write a polished blog post – both steps
using Groq's Llama 3.3 70B model.
```

```
"""
```

```
import os
from groq import Groq
```

```
from dotenv import load_dotenv
```

```
load_dotenv()
```

```
class BlogPostAgent:
```

```
    """
```

```
    Agent 2 – Researches a transcript internally, then writes a blog post.
```

```
    Internal flow:
```

- ```
    1. _research(transcript)          → research_notes (LLM call 1)
    2. _write(transcript, research_notes) → blog_post (LLM call 2)
```

```
    """
```

```
MODEL = "llama-3.3-70b-versatile"
```

```
def __init__(self):
```

```
    api_key = os.getenv("GROQ_API_KEY")
```

```
    if not api_key:
```

```
        raise EnvironmentError("GROQ_API_KEY not found.")
```

```
    self.client = Groq(api_key=api_key)
```

```
    print("[BlogPostAgent] Initialised ✓")
```

```
def run(self, transcript: str) -> str:
```

```
    self._validate_transcript(transcript)
```

```
    print(f"[BlogPostAgent] Received transcript ({len(transcript)} chars)")
```

```
    research_notes = self._research(transcript)
```

```
    blog_post = self._write(transcript, research_notes)
```

```
    print(f"[BlogPostAgent] Done – blog post written ({len(blog_post)} chars)")
```

```
    return blog_post
```

```
def _research(self, transcript: str) -> str:
```

```
    """LLM call 1 – analyse transcript, produce structured research notes."""
```

```
    print("[BlogPostAgent] Step 1/2 – Running internal research ...")
```

```
    system_prompt = """You are a senior content researcher and strategist.
```

```
Analyse the transcript and produce structured research notes covering:
```

- ```
1. CORE TOPIC, 2. TARGET AUDIENCE, 3. KEY INSIGHTS (4-6),
4. HOOK ANGLE, 5. SECTION STRUCTURE (3-5 headings),
6. HUMAN ELEMENT, 7. CLOSING DIRECTION."""
```

```
    user_prompt = f"Transcript:\n---\n{transcript}\n---\nProduce research notes now."
```

```
    response = self.client.chat.completions.create(
```

```
        model=self.MODEL,
```

```
        messages=[
```

```
            {"role": "system", "content": system_prompt},
```

```
            {"role": "user", "content": user_prompt},
```

```
        ],
```

```
        temperature=0.4,
```

```
        max_tokens=1500,
```

```
    )
```

```
    return response.choices[0].message.content.strip()
```

```
def _write(self, transcript: str, research_notes: str) -> str:
```

```
    """LLM call 2 – write a publish-ready blog post from notes + transcript."""
```

```
    print("[BlogPostAgent] Step 2/2 – Writing blog post ...")
```

```
    system_prompt = """You are an expert blog writer. Write in clean markdown.
```

```
Start with a # Title, use ## section headings, write in flowing paragraphs
```

```
(no bullet dumps). Target 600-900 words. Open with a hook, never with a
```

```
definition. End with a forward-looking thought, not a summary."""
```

```
    user_prompt = (
```

```
        f"Research notes:\n{research_notes}\n\n"
```

```
        f"Transcript excerpt:\n{transcript[:3000]}\n\n"
```

```
        "Write the complete blog post now."
```

```
    )
```

```
    response = self.client.chat.completions.create(
```

```
        model=self.MODEL,
```

```
        messages=[
```

```

        {"role": "system", "content": system_prompt},
        {"role": "user", "content": user_prompt},
    ],
    temperature=0.7,
    max_tokens=2000,
)
return response.choices[0].message.content.strip()

def _validate_transcript(self, transcript: str) -> None:
    if not transcript or not isinstance(transcript, str):
        raise ValueError("Transcript must be a non-empty string.")
    if len(transcript.strip()) < 100:
        raise ValueError("Transcript too short (minimum 100 characters).")

```

## orchestrator.py

---

### Location

yt-agent2/orchestrator.py

### Purpose

Thin coordinator layer that wires the two agents together. Instantiates both TranscriptAgent and BlogPostAgent on startup, then in run() calls them sequentially: first TranscriptAgent to get the transcript, then BlogPostAgent to produce the blog post. Returns a dict with both 'transcript' and 'blog\_post' keys to the Streamlit UI. Contains no business logic — all intelligence lives in the agents.

### Full Contents

```

"""
Orchestrator
-----
Coordinates TranscriptAgent and BlogPostAgent in sequence.
Receives a YouTube URL, returns a finished blog post.

Flow:
    YouTube URL → TranscriptAgent → transcript → BlogPostAgent → blog post
"""

from agents.transcript_agent import TranscriptAgent
from agents.blog_post_agent import BlogPostAgent

class Orchestrator:
    """
    Thin coordinator — holds no business logic of its own.
    Just wires the two agents together in the right order.
    """

    def __init__(self):
        print("[Orchestrator] Initialising agents ...")
        self.transcript_agent = TranscriptAgent()
        self.blog_post_agent = BlogPostAgent()
        print("[Orchestrator] Ready ✓")

    def run(self, youtube_url: str) -> dict:
        """
        Returns:
            dict with keys:
                'transcript' — raw transcript string
                'blog_post' — finished blog post in Markdown
        """
        print(f"\n[Orchestrator] Starting pipeline for: {youtube_url}")

        print("[Orchestrator] Calling TranscriptAgent ...")
        transcript = self.transcript_agent.run(youtube_url)
        print(f"[Orchestrator] Transcript received ({len(transcript)} chars)")

```

```

print("[Orchestrator] Calling BlogPostAgent ...")
blog_post = self.blog_post_agent.run(transcript)
print(f"[Orchestrator] Blog post received ({len(blog_post)} chars)")

print("[Orchestrator] Pipeline complete ✓")
return {"transcript": transcript, "blog_post": blog_post}

```

## app.py

---

### Location

yt-agent2/app.py

### Purpose

Streamlit web application — the user-facing entry point. Renders a URL input field and a 'Generate Blog Post' button. On submit it instantiates the Orchestrator inside a `st.status()` block (so the user sees live progress messages), runs the full pipeline, then displays results in two tabs: 'Blog Post' (rendered Markdown + download button) and 'Raw Transcript' (read-only text area). This is the file Railway runs.

### Full Contents

```

import streamlit as st
from orchestrator import Orchestrator

st.set_page_config(
    page_title="YouTube to Blog Post",
    page_icon="📄",
    layout="centered"
)

st.title("📄 YouTube to Blog Post")
st.caption("Paste a YouTube URL and get a publish-ready blog post in seconds.")

url = st.text_input(
    label="YouTube URL",
    placeholder="https://www.youtube.com/watch?v=...",
    help="Any public YouTube video with speech will work."
)

generate = st.button("Generate Blog Post", type="primary", use_container_width=True)

if generate:
    if not url.strip():
        st.warning("Please paste a YouTube URL first.")
        st.stop()

    try:
        with st.status("Running pipeline...", expanded=True) as status:
            st.write("Initialising agents...")
            orchestrator = Orchestrator()

            st.write("Downloading and transcribing audio...")
            original_run = orchestrator.transcript_agent.run

            def run_with_progress(youtube_url):
                result = original_run(youtube_url)
                st.write(f"Transcript ready – {len(result):,} characters")
                return result

            orchestrator.transcript_agent.run = run_with_progress

            st.write("Researching and writing blog post...")
            result = orchestrator.run(url.strip())

            status.update(label="Done!", state="complete", expanded=False)
    except Exception as e:

```

```

        st.error(f"Something went wrong: {e}")
        st.stop()

blog_post = result["blog_post"]
transcript = result["transcript"]

st.divider()
tab1, tab2 = st.tabs(["Blog Post", "Raw Transcript"])

with tab1:
    st.markdown(blog_post)
    st.divider()
    st.download_button(
        label="Download as .md",
        data=blog_post,
        file_name="blog_post.md",
        mime="text/markdown",
        use_container_width=True
    )

with tab2:
    st.text_area(
        label="Transcript",
        value=transcript,
        height=400,
        disabled=True
    )

```

## requirements.txt

---

### Location

yt-agent2/requirements.txt

### Purpose

Lists all Python package dependencies with minimum version pins. Used by Railway's build command (pip install -r requirements.txt) and by developers setting up locally. Note: ffmpeg is a system dependency handled separately via nixpacks.toml on Railway, or installed manually on local machines.

### Full Contents

```

# YouTube to Blog Post – dependencies
# Install: pip install -r requirements.txt

# Core agent dependencies
yt-dlp>=2024.1.0      # YouTube audio download
groq>=0.9.0          # Groq API client (Whisper + Llama)

# Utilities
python-dotenv>=1.0.0  # Load GROQ_API_KEY from .env

# UI
streamlit>=1.35.0     # Web interface

# ffmpeg is also required by yt-dlp for audio conversion.
# Install separately (not via pip):
#   Ubuntu/Railway: handled by nixpacks.toml (nixPkgs = ["ffmpeg"])
#   Mac:           brew install ffmpeg
#   Windows:       https://ffmpeg.org/download.html

```

## Procfile

---

### Location

yt-agent2/Procfile

### Purpose

Standard Heroku-style process file read by Railway to know how to start the web process. Passes \$PORT (injected by Railway at runtime) to Streamlit's --server.port flag, and binds to 0.0.0.0 so Railway's reverse proxy can route external traffic to the container.

### Full Contents

```
web: streamlit run app.py --server.port $PORT --server.address 0.0.0.0
```

## railway.toml

---

### Location

yt-agent2/railway.toml

### Purpose

Railway-specific deployment configuration. Tells Railway to use the Nixpacks builder (which reads nixpacks.toml for system packages), sets the pip install build command, duplicates the start command from the Procfile for explicitness, and configures the service to restart automatically on failure.

### Full Contents

```
[build]
builder = "nixpacks"
buildCommand = "pip install -r requirements.txt"

[deploy]
startCommand = "streamlit run app.py --server.port $PORT --server.address 0.0.0.0"
restartPolicyType = "on-failure"
```

## nixpacks.toml

---

### Location

yt-agent2/nixpacks.toml

### Purpose

Instructs Nixpacks (Railway's build system) to install ffmpeg as a system-level package during the build phase, before pip installs Python dependencies. ffmpeg is required by yt-dlp's FFmpegExtractAudio post-processor to convert downloaded video streams into MP3 files.

### Full Contents

```
[phases.setup]
nixPkgs = ["ffmpeg"]
```

## .env.example

---

### Location

yt-agent2/.env.example

### Purpose

Template showing which environment variables must be set for the app to run. Committed to version control so collaborators know what secrets to supply. The actual .env file (containing the real key) is gitignored and never committed. On Railway, GROQ\_API\_KEY is set as a service variable in the Railway dashboard rather than via a .env file.

### Full Contents

```
# Copy this file to .env and fill in your key.
# Get your free key at: https://console.groq.com
```

```
GROQ_API_KEY=your_groq_api_key_here
```